KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NAAC, Approved by AICTE, Affiliated to JNTUH, Narayanguda, Hyderabad – 500029



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

LAB RECORD
WEB AND SOCIAL MEDIA ANALYTICS LAB

B.Tech. IV YEAR I SEM (KR23)

ACADEMIC YEAR 2026-27

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

Certificate

This is to certify that following is a Bonafide Record of the workbook task done by

_____bearing Roll No_____ of _____

Branch of _____ year B.Tech Course in the_____

Subject during the Academic year _____& _____under our supervision.

Number of week tasks completed: _____

Signature of Staff Member Incharge

Signature of Head of the Dept.

Signature of Internal Examiner

Signature of External Examiner

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

Daily Laboratory Assessment Sheet

Name of the Lab: Name of the Student:

Class: HT. No:

S.No.	Name of the Experiment	Date	Observation Marks (3M)	Record Marks (4M)	Viva Voice Marks (3M)	Total Marks (10M)	Signature of Faculty
	TOTAL						

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029

Department of Computer Science & Engineering (DS)

Vision of the Institution:

To be the fountain head of latest technologies, producing highly skilled, globally competent engineers.

Mission of the Institution:

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepare students to become successful professionals.
- To establish Industry Institute Interaction to make students ready for the industry.
- To provide exposure to students on latest hardware and software tools.
- To promote research based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students

Vision & Mission of Department

Vision of the Department

To be among the region's premier teaching and research Computer Science and Engineering departments producing globally competent and socially responsible graduates in the most conducive academic environment.

Mission of the Department

- To provide faculty with state of the art facilities for continuous professional development and research, both in foundational aspects and of relevance to emerging computing trends.
- To impart skills that transform students to develop technical solutions for societal needs and inculcate entrepreneurial talents.
- To inculcate an ability in students to pursue the advancement of knowledge in various specializations of Computer Science and Engineering and make them industry-ready.
- To engage in collaborative research with academia and industry and generate adequate resources for research activities for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To cultivate responsibility through sharing of knowledge and innovative computing solutions that benefits the society-at-large.
- To collaborate with academia, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities.

Department of Computer Science & Engineering(DS)

PROGRAM OUTCOMES (POs)

PO1:Engineering Knowledge

Apply the knowledge of mathematics, natural science, computing, and engineering fundamentals and an engineering specialisation to the solution of complex engineering problems.

PO2:Problem Analysis

Identify, formulate, and analyse complex engineering problems, reaching substantiated conclusions with consideration for the holistic nature of the problem.

PO3:Design/Development of Solutions

Design creative solutions for complex engineering problems and design systems, components, or processes to meet identified needs — with consideration for public health and safety, and cultural, societal, and environmental factors. Sustainability is now intrinsic to design.

PO4:Conduct Investigations of Complex Problems

Conduct investigations of complex engineering problems using research-based knowledge and research methods, including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.

PO5:Engineering Tool Usage

Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling — recognising their limitations — to solve complex engineering problems. (Previously titled "Modern Tool Usage.")

PO6:The Engineer and the World

Analyse and evaluate societal and environmental aspects while solving complex engineering problems, considering their impact on sustainability with reference to economy, health, safety, legal frameworks, culture, and the environment.

PO7:Ethics

Apply ethical principles and commit to professional ethics, human values, diversity, and inclusion; and adhere to relevant national and international laws. (Diversity and inclusion are now explicit additions.)

PO8:Individual and Collaborative Team Work

Function effectively as an individual, and as a member or leader in diverse and multi-disciplinary teams.

PO9:Communication

Communicate effectively and inclusively within the engineering community and with society at large — comprehending and writing effective reports and design documentation, and making effective presentations — considering cultural, language, and learning differences.

PO10:Project Management and Finance

Apply knowledge and understanding of engineering management principles and economic decision-making, and apply these to one's own work, as a member and leader in a team, to manage projects in multi-disciplinary environments.

PO11:Life-Long Learning

Recognise the need for, and have the preparation and ability for, independent and life-long learning, adaptability to new and emerging technologies, and critical thinking in the broadest context of technological change.

.

Department of Computer Science & Engineering (DS)

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Computer Science solutions for social upliftments.

PSO2: Shall have expertise on the evolving technologies like Mobile Apps, CRM, ERP, Big Data, etc.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will have successful careers in computer related engineering fields or will be able to successfully pursue advanced higher education degrees.

PEO2: Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

PEO3: Graduates will engage in life-long learning and professional development by rapidly adapting changing work environment.

PEO4: Graduates will communicate effectively, work collaboratively and exhibit high levels of professionalism and ethical responsibility.

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NAAC, Approved by AICTE, Affiliated to JNTUH, Narayanguda, Hyderabad – 500029

B.Tech. in COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

IV Year I Semester Syllabus (KR23)

WEB AND SOCIAL MEDIA ANALYTICS LAB (21CD732PE)

Pre-requisites/ Co-requisites:	L	T	P	C
1. PP207ES:Python Programming Lab Course	3	0	0	3
2. 21CS306PC: Machine Learning Using Python Lab				

Course Objectives: The course will help to

1. Learn decision support systems.
2. Learn language processing concepts on text analytics.
3. Learn sentiment analysis.
4. Learn Web Analytics, Web Mining
5. Learn search engine optimization and web analytics.

Course Outcomes: After learning the concepts of this course, the student is able to

1. Understand decision support systems.
2. Apply natural language processing concepts on text analytics.
3. Understand sentiment analysis.
4. Understand Web Analytics, Web Mining
5. Knowledge on search engine optimization and web analytics.

List of Experiments

1. Preprocessing text document using NLTK of Python
a. Stop word elimination
b. Stemming
c. Lemmatization
d. POS tagging
e. Lexical analysis
2. Sentiment analysis on customer review on products
3. Web analytics
a. Web usage data (web server log data, clickstream analysis)
b. Hyperlink data
4. Search engine optimization- implements spamdexing
5. Use Google analytics tools to implement the following
a. Conversion Statistics
b. Visitor Profiles
6. Use Google analytics tools to implement the Traffic Sources.
7. Data Processing Techniques:
(i) Data cleaning (ii) Data transformation – Normalization (iii) Data integration
8. Implementation of Decision Tree algorithm

9. Classification of data using logistic regression
10. Classification of data using K – nearest neighbor approach
11. Implementation of K – means algorithm.

Resources:
1. Stanford core NLP package
2. GOOGLE.COM/ANALYTICS

TEXT BOOKS:

1. Ramesh Sharda, Dursun Delen, Efraim Turban, BUSINESS INTELLIGENCE AND ANALYTICS: SYSTEMS FOR DECISION SUPPORT, Pearson Education.

REFERENCE BOOKS:

1. Rajiv Sabherwal, Irma Becerra- Fernandez," Business Intelligence –Practice, Technologies and Management", John Wiley 2011.

2. Lariss T. Moss, Shaku Atre, "Business Intelligence Roadmap", Addison-Wesley It Service.

3. Yuli Vasiliev, "Oracle Business Intelligence: The Condensed Guide to Analysis and Reporting", SPD Shroff, 2012.

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTE)

Accredited by NAAC, Approved by AICTE, Affiliated to JNTUH, Narayanguda, Hyderabad – 500029

Department of Computer Science & Engineering (DS)

Course Outcomes and CO-PO-PSO Mapping

Course Outcomes:

After learning the contents of this course, the student is able to

CO1	Understand decision support systems.
CO2	Apply natural language processing concepts on text analytics.
CO3	Understand sentiment analysis
CO4	Understand Web Analytics, Web Mining
CO5	Knowledge on search engine optimization and web analytics.

CO-PO-PSO MAPPING:

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO-1	PSO-2
CO1	3				1							1		1
CO2	3	3	3	2	1							1	1	
CO3	3	3	3		1							2	1	
CO4	2	3	3	1	1							1	1	
CO5	3	2	1		1							1		1

Experiment 1: Preprocessing Text Document

Aim: Preprocessing text document using NLTK of Python
a. Stop word elimination
b. Stemming
c. Lemmatization
d. POS tagging
e. Lexical analysis

Software Required: Google Co Lab, Jupyter notebook, Kaggle .

Theory:

Natural Language Toolkit (NLTK) is one of the largest Python libraries for performing various Natural Language Processing tasks. From rudimentary tasks such as text pre-processing to tasks like vectorized representation of text – NLTK's API has covered everything. In this article, we will accustom ourselves to the basics of NLTK and perform some crucial NLP tasks: Tokenization, Stemming, Lemmatization, and POS Tagging.

Tokenization Tokenization refers to break down the text into smaller units. It entails splitting paragraphs into sentences and sentences into words. It is one of the initial steps of any NLP pipeline. Let us have a look at the two major kinds of tokenization that NLTK provides:

Work Tokenization It involves breaking down the text into words.

Stemming: Reduce words to their root form using algorithms like the Porter Stemmer.For typical stemming results using Porter Stemmer, "running" will be reduced to "run," which is the expected behavior.

Lemmatization: Reduce words to their base or dictionary form using the WordNet Lemmatizer.

POS (Part-of-Speech) tags represent the grammatical categories of words in a sentence. Each tag denotes a specific role that a word plays within the structure of a sentence. Here's a brief explanation of the POS tags you mentioned:

Common POS Tags

1. **NNP (Proper Noun, Singular):**

Represents names of specific people, places, or organizations.
Example: "NLTK" and "Python" in your example.

2. **VBZ (Verb, 3rd Person Singular Present):**

Indicates a verb in the present tense that is used with third-person singular subjects.
Example: "is" in your example.

3. **DT (Determiner):**

Used to introduce nouns and specify which noun is being referred to.

Example: "a" in your example.

4. *NN (Noun, Singular or Mass):*

Represents a singular noun that refers to a general, non-specific item or concept.
Example: "platform" in your example.

5. *VBG (Verb, Gerund or Present Participle):*

Represents a verb form ending in -ing that functions as a noun or as part of a continuous tense.
Example: "leading" and "building" in your example.

6. *IN (Preposition or Subordinating Conjunction):*

Represents words used to show relationships between other words or phrases.
Example: "for" in your example.

7. *NNS (Noun, Plural):*

Represents a noun in its plural form.
Example: "programs" in your example.

E. Lexical Analysis: Breaks text into tokens for analysis.Tokens are the individual units into which text is broken down during the process of text analysis. In natural language processing (NLP), tokens are typically the smallest meaningful elements of text and can be words, phrases, or even punctuation marks

Code:

Installation: NLTK can be installed simply using pip or by running the following code.

```
! pip install nltk
import nltk
nltk.download('all')
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download necessary resources
nltk.download('stopwords')
nltk.download('punkt')
```

Tokenization

```
# Tokenization using NLTK
```

```
from nltk import word_tokenize, sent_tokenize
sent = "WEB AND SOCIAL MEDIA ANALYTICS LAB .\
Preprocessing text document using NLTK of Python."
print(word_tokenize(sent))

+print(sent_tokenize(sent))
```

OUTPUT:

```
['WEB', 'AND', 'SOCIAL', 'MEDIA', 'ANALYTICS', 'LAB', '.Preprocessing', 'text', 'document', 'using', 'NLTK', 'of', 'Python', '.']
['WEB AND SOCIAL MEDIA ANALYTICS LAB .Preprocessing text document using NLTK of Python.']
```

A. Stop Word Elimination: Remove common words that don't contribute significant meaning.

```
# Sample text
text = "This is a sample text document."

# Tokenize and remove stop words
words = word_tokenize(text)
filtered_words = [word for word in words if word.lower() not in stopwords.words('english')]

print("Filtered words:", filtered_words)
```

OUTPUT:

```
Filtered words: ['sample', 'text', 'document', '.']
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

B. Stemming: Reduce words to their root form using algorithms like the Porter Stemmer.For typical stemming results using Porter Stemmer, "running" will be reduced to "run," which is the expected behavior.

```
from nltk.stem import PorterStemmer

# Create stemmer object
stemmer = PorterStemmer()

# Sample words
words = ["running", "runs", "ran"]

# Apply stemming
stemmed_words = [stemmer.stem(word) for word in words]

print("Stemmed words:", stemmed_words)
```

OUTPUT:

```
Stemmed words: ['run', 'run', 'ran']
```

C. Lemmatization: Reduce words to their base or dictionary form using the WordNet Lemmatizer.

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
from nltk.tokenize import word_tokenize
import nltk

# Download necessary resources
nltk.download('wordnet')
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
from nltk.stem import WordNetLemmatizer
# create an object of class WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print(lemmatizer.lemmatize("plays", 'v'))
print(lemmatizer.lemmatize("played", 'v'))
print(lemmatizer.lemmatize("play", 'v'))
print(lemmatizer.lemmatize("playing", 'v'))
```

OUTPUT:

```
play
play
play
play
```

D. POS Tagging: Label words with their part of speech.

```
import nltk
from nltk.tokenize import word_tokenize

# Download necessary resources
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt')
# Sample text
text = "NLTK is a leading platform for building Python programs."

# Tokenize and POS tagging
words = word_tokenize(text)
pos_tags = nltk.pos_tag(words)

print("POS tags:", pos_tags)
```

OUTPUT:

```
POS tags: [('NLTK', 'NNP'), ('is', 'VBZ'), ('a', 'DT'), ('leading', 'VBG'), ('platform', 'NN'), ('for', 'IN'), ('building', 'VBG'), ('Python', 'NNP'), ('programs', 'NNS'), ('.', '.')]
```

E. Lexical Analysis: Breaks text into tokens for analysis.Tokens are the individual units into which text is broken down during the process of text analysis. In natural language processing (NLP), tokens are typically the smallest meaningful elements of text and can be words, phrases, or even punctuation marks

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

# Download necessary resources
nltk.download('punkt')
```

OUTPUT:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
True
```

```
# Sample text
text = "NLTK is a leading platform. It provides easy-to-use interfaces."

# Tokenize sentences and words
sentences = sent_tokenize(text)
words = word_tokenize(text)

print("Sentences:", sentences)
print("Words:", words)
```

OUTPUT:

```
Sentences: ['NLTK is a leading platform.', 'It provides easy-to-use interfaces.']
Words: ['NLTK', 'is', 'a', 'leading', 'platform', '.', 'It', 'provides', 'easy-to-use', 'interfaces', '.']
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

#Using spacy

```
import spacy
# Load the small English model
nlp = spacy.load("en_core_web_sm")
# Sample text
text = "spaCy is a powerful NLP library."
# Process the text
doc = nlp(text)
# Extract and print tokens and their POS tags
for token in doc:
```

```
print(f"{token.text}: {token.pos_}")
```

Output:

```
spaCy: INTJ
is: AUX
a: DET
powerful: ADJ
NLP: PROPN
library: NOUN
.: PUNCT
```

Experiment 2: Sentiment analysis on customer review on products

Aim: Sentiment analysis on customer review on products

Software Required: Google Co Lab, Jupyter notebook, Kaggle.

Theory:

- Sentiment analysis is trying to answer the question "What do people feel about a certain topic?" by digging into opinions of many using a variety of automated tools. **Sentiment analysis** has many names. It's often referred to as opinion mining, subjectivity analysis, and appraisal extraction, with some connections to affective computing (computer recognition and expression of emotion). **In a business**, especially in marketing and customer relationship management, sentiment analysis seeks to detect favorable and unfavorable opinions toward specific products and/or services using large numbers of textual data sources (customer feedback in the form of Web postings , tweets, blogs, etc.).
- **Sentiment that appears in text comes in two flavors**: explicit, where the subjective sentence directly expresses an opinion ("It's a wonderful day"), and implicit, where the text implies an opinion ("The handle breaks too easily").The following applications of sentiment analysis are meant to illustrate the power and the widespread coverage of this technology

1. Voice of the customer (VOC)
2. Voice of the Market (VOM)
3. Voice of the Employee (VOE)
4. Brand Management
5. Financial Markets
6. Politics
7. Government Intelligence

Code:

```
#Install the necessary libraries:
!pip install nltk
#Download NLTK resources (only once):
import nltk
nltk.download('vader_lexicon')



# Import necessary libraries
from nltk.sentiment import SentimentIntensityAnalyzer
```

```
# Initialize the Sentiment Analyzer
sia = SentimentIntensityAnalyzer()

# Sample customer reviews
reviews = [
    "This product is amazing! I love it.",
    "The quality is bad, and I am very disappointed.",
    "It works okay, but it's not the best.",
    "I absolutely hate this product. It's terrible!",
    "Pretty good product for the price."
]

# Analyze sentiment for each review
for review in reviews:
    sentiment_score = sia.polarity_scores(review)
    print(f"Review: {review}")
    print(f"Sentiment Scores: {sentiment_score}\n")
```

OUTPUT:

```
Requirement already satisfied: nltk in /usr/local/lib/python3.10/dist-packages (3.8.1)
Requirement already satisfied: click in /usr/local/lib/python3.10/dist-packages (from nltk) (8.1.7)
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (from nltk) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.10/dist-packages (from nltk) (2024.9.11)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from nltk) (4.66.5)
Review: This product is amazing! I love it.
Sentiment Scores: {'neg': 0.0, 'neu': 0.325, 'pos': 0.675, 'compound': 0.8516}

Review: The quality is bad, and I am very disappointed.
Sentiment Scores: {'neg': 0.535, 'neu': 0.465, 'pos': 0.0, 'compound': -0.7841}

Review: It works okay, but it's not the best.
Sentiment Scores: {'neg': 0.379, 'neu': 0.5, 'pos': 0.121, 'compound': -0.6251}

Review: I absolutely hate this product. It's terrible!
Sentiment Scores: {'neg': 0.649, 'neu': 0.351, 'pos': 0.0, 'compound': -0.8118}

Review: Pretty good product for the price.
Sentiment Scores: {'neg': 0.0, 'neu': 0.396, 'pos': 0.604, 'compound': 0.7269}

[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
```

```
review = "The quality is bad, and I am very disappointed."
sentiment_score = sia.polarity_scores(review)
print(sentiment_score)
```

Output:

```
{'neg': 0.535, 'neu': 0.465, 'pos': 0.0, 'compound': -0.7841}
```

Experiment 3: Web analytics

Aim: Web analytics
a. Web usage data (web server log data, clickstream analysis)
b. Hyperlink data

Software Required: Google Co Lab, Jupyter notebook, Kaggle.

Theory:

Web Usage Data:

- **Definition**: Tracks user activities on a website using server logs, including page visits, timestamps, and IP addresses.
    - **Clickstream Analysis**: Studies the sequence of pages users visit to understand navigation patterns and user behavior.
    - **Tools**: Uses web server logs, Google Analytics, and log file parsers for detailed analysis.

Hyperlink Data:

- **Definition**: Focuses on the connections between web pages using hyperlinks to analyze website structure.
- **Visualization**: Often represented as a graph where nodes are pages and edges are hyperlinks.
- **SEO Insights**: Assists in identifying link structures that improve search engine rankings.
- **User Navigation**: Reveals internal linking effectiveness and navigation challenges.
- **Applications**: Supports website design optimization, crawling strategies, and content discoverability.

Code:

```
#Step 1: Upload the Log File
#Upload a web server log file (e.g., web_server_log.txt).
# File upload in Colab
from google.colab import files

uploaded = files.upload()   # Upload the web server log file
```

```
#Step 2: Process Web Usage Data
#This step extracts information such as IP address, timestamps, and
requested URLs.
import pandas as pd
import re

# Load the log file
log_file = list(uploaded.keys())[0]   # Get uploaded file name
with open(log_file, 'r') as file:
    log_data = file.readlines()

# Define a function to parse web server logs
def parse_log_line(line):
    # Regex to match log lines (Example: Common Apache Log Format)
    log_pattern = r'(\d+\.\d+\.\d+\.\d+) - - \[(.*?)\] "(GET|POST)
(.*?) HTTP/.*?" (\d+) (\d+)'
    match = re.match(log_pattern, line)
    if match:
        return {
            'IP Address': match.group(1),
            'Timestamp': match.group(2),
            'Method': match.group(3),
            'URL': match.group(4),
            'Status Code': int(match.group(5)),
            'Bytes Transferred': int(match.group(6))
        }
    return None

# Parse the log data
#Parsing means analyzing and converting raw data or text into a
structured format that is easier to process and understand.
parsed_data = [parse_log_line(line) for line in log_data if
parse_log_line(line)]
df_logs = pd.DataFrame(parsed_data)

# Display the parsed data
print("Parsed Web Server Logs:")
print(df_logs.head())
```

Output:

```
Parsed Web Server Logs:
      IP Address                Timestamp Method          URL  \
0  192.168.1.1  03/Dec/2024:10:00:00 +0000    GET     /index.html
1  192.168.1.1  03/Dec/2024:10:01:00 +0000    GET     /about.html
2  192.168.1.1  03/Dec/2024:10:02:00 +0000    GET   /contact.html
3  192.168.1.2  03/Dec/2024:10:00:30 +0000    GET     /index.html
4  192.168.1.2  03/Dec/2024:10:01:30 +0000    GET  /products.html

   Status Code  Bytes Transferred
0          200               1024
1          200                512
2          200                256
3          200               1024
4          200               2048
```

```
#Step 3: Perform Clickstream Analysis
#Analyze user behavior based on the sequence of URLs visited.
# Analyze clickstream patterns
clickstream = df_logs.groupby('IP
Address')['URL'].apply(list).reset_index()

print("\nClickstream Data (Sequence of URLs visited by each
user):")
for _, row in clickstream.iterrows():
    print(f"User {row['IP Address']} visited: {',
'.join(row['URL'])}")
```

OUTPUT

```
Clickstream Data (Sequence of URLs visited by each user):
User 192.168.1.1 visited: /index.html, /about.html, /contact.html
User 192.168.1.2 visited: /index.html, /products.html
User 192.168.1.3 visited: /index.html, /services.html, /about.html
```

```
#Step 4: Analyze Hyperlink Data
import networkx as nx
```

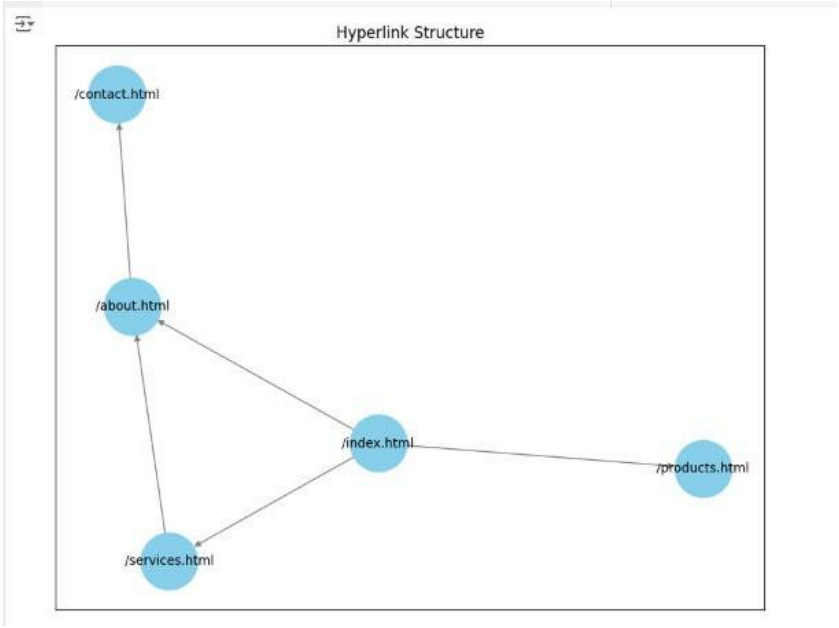
```
import matplotlib.pyplot as plt

# Extract hyperlink relationships (e.g., referer and target URL)
hyperlinks = [(df_logs.iloc[i]['URL'], df_logs.iloc[i+1]['URL'])
              for i in range(len(df_logs) - 1) if
df_logs.iloc[i]['IP Address'] == df_logs.iloc[i+1]['IP Address']]

# Create a graph using NetworkX
graph = nx.DiGraph()
graph.add_edges_from(hyperlinks)

# Plot the hyperlink structure
plt.figure(figsize=(10, 8))
nx.draw_networkx(graph, with_labels=True, node_color='skyblue',
edge_color='gray', node_size=2000, font_size=10)
plt.title("Hyperlink Structure")
plt.show()
```

OUTPUT:



Experiment 4: Search engine optimization- implements spamdexing

Aim: Search engine optimization- implements spamdexing

Software Required: Google Co Lab, Jupyter notebook, Kaggle .

Theory:

Spamdexing, or **search engine spamming**, refers to unethical or manipulative techniques used to improve a website's ranking on search engines through methods that violate search engine guidelines. These techniques often focus on tricking search engines into ranking a page higher than it should be, rather than improving the page's content or quality.

While **Search Engine Optimization (SEO)** is a legitimate practice aimed at improving a website's visibility through ethical means, **spamdexing** employs tactics that artificially inflate a site's ranking.

Code:

```
# Install required packages
!pip install requests beautifulsoup4

import requests
from bs4 import BeautifulSoup

# Function to get SEO-related data from a webpage
def analyze_seo(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')

    # Fetch title tag
    title = soup.title.string if soup.title else 'No title found'

    # Fetch meta description
    meta_description = soup.find('meta', {'name': 'description'})
    meta_description = meta_description['content'] if
meta_description else 'No meta description found'

    # Fetch all H1 tags
    h1_tags = [h1.get_text() for h1 in soup.find_all('h1')]

    print("SEO Analysis for:", url)
```

```
    print("Title:", title)
    print("Meta Description:", meta_description)
    print("H1 Tags:", h1_tags)

# Example URL
url = 'https://example.com'
analyze_seo(url)
```

Output:

```
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (2.32.3)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.10/dist-packages (4.12.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests) (2024.8.30)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.10/dist-packages (from beautifulsoup4) (2.6)
SEO Analysis for: https://example.com
Title: Example Domain
Meta Description: No meta description found
H1 Tags: ['Example Domain']
```

Experiment 5: Google analytics tools to implement Conversion Statistics and Visitor Profiles

Aim: Use Google analytics tools to implement the following
a. Conversion Statistics
b. Visitor Profiles

Software Required: Google Co Lab, Jupyter notebook, Google Analytics .

Theory:

Google Analytics is a powerful web analytics tool that helps track and report website traffic and user behavior.It provides insights into audience demographics, traffic sources, and key metrics like bounce rate, session duration, and conversions.Businesses use Google Analytics to optimize website performance, improve user experience, and make data-driven decisions.

Conversion Statistics

Definition:

Conversion statistics measure how many visitors complete a desired action (or **conversion**) on your website. A "conversion" can refer to any action you consider valuable, such as:

- Making a purchase.
- Signing up for a newsletter.
- Downloading an eBook.
- Filling out a contact form.
- Watching a video.

These statistics give you insight into how effectively your website drives users toward these desired actions.

Key Metrics in Conversion Statistics:

- **Conversion Rate**: The percentage of visitors who complete a specific goal out of the total visitors.
  - Formula: `(Conversions / Total Visitors) × 100`
- **Goal Completions**: Total number of completed goals, like purchases or sign-ups.
- **Revenue (for e-commerce)**: Total income generated from sales tracked on your website.
- **Abandonment Rate**: The percentage of users who begin but do not complete a conversion process (e.g., adding items to a cart but not checking out).

2. Visitor Profiles

Definition:

Visitor profiles are detailed descriptions of your website visitors based on their behavior, demographics, interests, and other attributes. This analysis helps you understand **who your visitors are**, **where they come from**, and **how they interact with your website**.

Components of Visitor Profiles:

- **Demographics**: Age, gender, and other demographic information.
- **Interests**: Topics or categories that users are interested in (e.g., sports, technology, fashion).
- **Geolocation**: Country, city, or region of your visitors.
- **Device and Technology**: Devices (mobile, desktop, tablet), browsers, and operating systems used by visitors.
- **Traffic Source**: How visitors arrive at your website (e.g., organic search, paid ads, social media).
- **Behavior**: How visitors interact with your website (e.g., time spent on site, pages visited, bounce rate).

Steps in Google Analytics

Google provides a free demo account for practicing and exploring Google Analytics features. This account is linked to Google's own merchandise store, allowing you to work with real-world data.

How to Access the Demo Account:

1. Go to Google Analytics Demo Account.
2. Sign in with your Google account and access the demo account.
3. Explore **Conversion Statistics** under **Conversions** > **Goals** and **E-commerce** sections.
4. Analyze **Visitor Profiles** under **Audience** > **Demographics**, **Interests**, and **Geo**.

Code:

```
import pandas as pd
import random

# Simulated Conversion Statistics Data
conversion_data = {
    "Date": pd.date_range(start="2023-11-01", periods=30).strftime("%Y-%m-
%d"),
    "Goal Completions": [random.randint(50, 200) for _ in range(30)],
    "Transactions": [random.randint(10, 100) for _ in range(30)],
```

```
}

# Simulated Visitor Profiles Data
visitor_data = {
    "Country": ["USA", "India", "Germany", "UK", "Canada"],
    "Age Group": ["18-24", "25-34", "35-44", "45-54", "55+"],
    "Gender": ["Male", "Female", "Other"],
    "Users": [random.randint(1000, 5000) for _ in range(15)],
}

# Create DataFrames
conversion_df = pd.DataFrame(conversion_data)
visitor_df = pd.DataFrame(
    [
        {"Country": c, "Age Group": a, "Gender": g, "Users":
random.randint(100, 500)}
        for c in visitor_data["Country"]
        for a in visitor_data["Age Group"]
        for g in visitor_data["Gender"]
    ]
)

# Display the DataFrames
print("Simulated Conversion Statistics:")
print(conversion_df)

print("\nSimulated Visitor Profiles:")
print(visitor_df)
```

Output:

```
Simulated Conversion Statistics:
          Date  Goal Completions  Transactions
0   2023-11-01               192            36
1   2023-11-02               148            83
2   2023-11-03               106            61
3   2023-11-04               187            50
4   2023-11-05                68            15
5   2023-11-06                63            59
6   2023-11-07               183            23
7   2023-11-08               191            17
8   2023-11-09                66            97
9   2023-11-10               141            57
10  2023-11-11                52            81
11  2023-11-12               192            53
12  2023-11-13               166            88
13  2023-11-14               115            15
14  2023-11-15               165            24
15  2023-11-16                70            69
16  2023-11-17                65            59
17  2023-11-18                52            90
18  2023-11-19               197            41
19  2023-11-20               184            75
20  2023-11-21               133            15
21  2023-11-22                95            53
22  2023-11-23               101            98
23  2023-11-24               170            40
24  2023-11-25               167            55
25  2023-11-26                88            68
26  2023-11-27               103            56
27  2023-11-28                57            21
28  2023-11-29               119           100
29  2023-11-30               149            20

Simulated Visitor Profiles:
   Country Age Group  Gender  Users
0      USA     18-24    Male    300
1      USA     18-24  Female    128

3      USA     25-34    Male    463
4      USA     25-34  Female    364
```

Experiment 6: Use Google analytics tools to implement the Traffic Sources.

Aim Use Google analytics tools to implement the following Conversion Statistics

Software Required: Google Co Lab, Jupyter notebook, Kaggle ,Google Analytics .

Theory:

Using Google Analytics to implement and analyze **Traffic Sources** involves configuring your account, collecting data, and interpreting reports.

1. Set Up Google Analytics

Create a Google Analytics Account: If not done already, sign up at Google Analytics.
Add Tracking Code:
Navigate to `Admin` > `Data Streams` > `Web`.
Copy the Global Site Tag (gtag.js) and add it to the `<head>` section of your website's HTML.

2. Enable Traffic Source Tracking

Google Analytics automatically tracks traffic sources (organic, direct, referral, social). To ensure accuracy:
Use UTM parameters for your campaigns.

Example:

`?utm_source=google&utm_medium=email&utm_campaign=holiday_sale`.

Use tools like Google's Campaign URL Builder to generate UTM links.

Understand Traffic Source Categories

Traffic sources in Google Analytics are grouped as:
Organic Search: Visitors from search engines (e.g., Google, Bing).
Direct: Visitors who type your URL directly or bookmark it.
Referral: Visitors from other websites linking to yours.
Social: Traffic from social media platforms.
Paid Search/Advertising: Visitors from Google Ads or other campaigns.

View Traffic Source Reports

Log in to Google Analytics.
Navigate to:

`Reports` > `Acquisition` > `Traffic Acquisition` (GA4).

`Acquisition` > `All Traffic` > `Channels` (Universal Analytics).

Analyze metrics like:

Sessions: Number of visits.
Users: Number of unique visitors.
Bounce Rate: Percentage of single-page sessions.

Conversions: Traffic that completed desired actions (e.g., form submissions).

Segment Traffic Sources

To better understand performance:
Apply filters to view specific traffic sources.
Segment by **geography, device, or campaign.**

Optimize for Key Traffic Sources

Organic Search: Focus on SEO (keywords, content optimization).
Social Media: Post regularly with links to your site.
Referral: Build backlinks from reputable sites.
Paid Ads: Set up Google Ads campaigns and link them to Analytics.

Monitor and Report

Set goals in Analytics (`Admin` > `Goals`) to track conversions.
Generate regular reports to assess traffic source effectiveness and adjust strategies.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Simulate traffic data
np.random.seed(42)

# Traffic sources
sources = ['Organic Search', 'Direct', 'Referral', 'Social Media', 'Paid Search']
days = 30  # Number of days for simulation

# Generate random traffic data
data = {source: np.random.randint(50, 500, size=days) for source in sources}
data['Date'] = pd.date_range(start='2024-01-01', periods=days)

# Convert to DataFrame
traffic_df = pd.DataFrame(data)

# Visualization
plt.figure(figsize=(12, 6))
for source in sources:
    plt.plot(traffic_df['Date'], traffic_df[source], label=source)

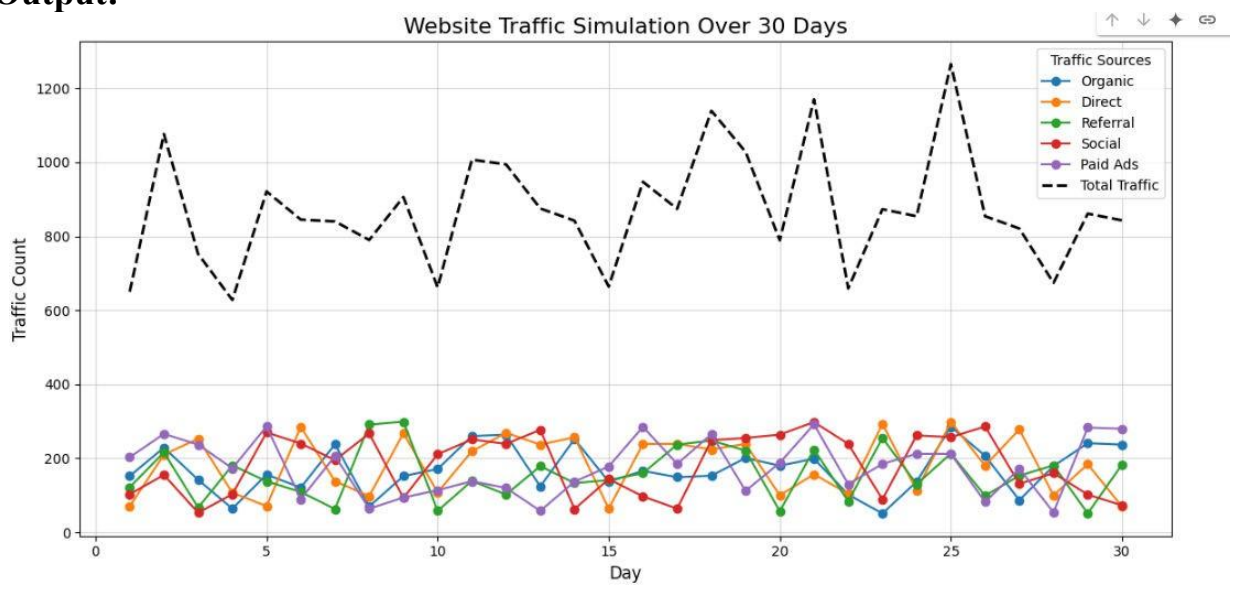
# Customize the plot
plt.title('Simulated Traffic Sources Over Time', fontsize=16)
```

```
plt.xlabel('Date', fontsize=12)
plt.ylabel('Traffic Volume', fontsize=12)
plt.legend(title='Traffic Source')
plt.grid(True)
plt.tight_layout()

# Show the plot
plt.show()

# Display sample data
print("Sample Simulated Traffic Data:")
print(traffic_df.head())
```

Output:



Experiment 7 : Data Processing Techniques

Aim: Data Processing Techniques:
(i) Data cleaning (ii) Data transformation – Normalization (iii) Data integration

Software Required: Google Co Lab, Jupyter notebook, Kaggle .

Theory:

Data processing refers to the process of collecting, organizing, and transforming raw data into a usable format for analysis, decision-making, or further processing. It involves a series of steps to ensure the data is accurate, clean, and structured, enabling meaningful insights.

Key stages of data processing include:

1. **Data Collection**: Gathering data from various sources (e.g., databases, sensors, or user input).
2. **Data Cleaning**: Removing errors, handling missing values, and ensuring consistency.
3. **Data Transformation**: Converting data into a suitable format or structure (e.g., normalization, encoding).
4. **Data Analysis**: Applying analytical methods to extract insights.
5. **Data Storage**: Storing processed data in databases or files for easy retrieval.
6. **Data Visualization**: Presenting data in charts, graphs, or reports for interpretation.

Data Processing Techniques

(i) Data Cleaning

1. **Handling Missing Values**: Data cleaning involves addressing missing or null values by either filling them with a predefined value (mean, median, or mode) or removing the rows/columns with missing data. This ensures consistency and reliability in the dataset.
2. **Removing Duplicates**: Duplicate records are eliminated to ensure that each data point is unique, preventing any biases or distortions in analysis or model performance.

(ii) Data Transformation – Normalization

1. **Rescaling Features**: Normalization is the process of rescaling numeric data to a standard range, typically [0, 1], which helps improve the performance of machine learning algorithms, especially those sensitive to the scale of data (like neural networks).
2. **Improving Model Performance**: By transforming the data into a uniform scale, normalization helps algorithms converge faster and avoid biased results, ensuring that no feature dominates the learning process due to its scale.

(iii) Data Integration

1. **Combining Data from Multiple Sources**: Data integration involves merging data from different sources (e.g., databases, APIs, or files) into a unified dataset. This allows for a comprehensive view and is essential for analysis and decision-making.
2. **Consistency Across Datasets**: During integration, it's crucial to ensure that the data from different sources aligns in terms of format, units, and types, thus maintaining consistency for further analysis.

Code:

1. Data Cleaning Handling missing values and converting categorical columns to appropriate formats.

```
import pandas as pd

# Load the Toyota dataset
toyotadf = pd.read_csv('Toyota.csv')   # Replace with your file path

# Display the initial dataset information
print("Initial Data Info:")
toyotadf.info()

# Handle missing values:
# For numeric columns (Age, MetColor), fill with mean or median
toyotadf['Age'] = toyotadf['Age'].fillna(toyotadf['Age'].mean())   #
Mean for Age
toyotadf['MetColor'] =
toyotadf['MetColor'].fillna(toyotadf['MetColor'].mean())    # Mean
for MetColor

# For categorical columns (FuelType), fill with 'Unknown'
toyotadf['FuelType'] = toyotadf['FuelType'].fillna('Unknown')

# Convert 'KM' and 'HP' columns to numeric, handling errors (e.g.,
converting 'object' type to numeric)
toyotadf['KM'] = pd.to_numeric(toyotadf['KM'], errors='coerce')
toyotadf['HP'] = pd.to_numeric(toyotadf['HP'], errors='coerce')

# Remove duplicates
toyotadf.drop_duplicates(inplace=True)
```

```
# Display cleaned data info
print("\nCleaned Data Info:")
toyotadf.info()
```

```
Initial Data Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1436 entries, 0 to 1435
Data columns (total 11 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   sno         1436 non-null   int64
 1   Price       1436 non-null   int64
 2   Age         1336 non-null   float64
 3   KM          1436 non-null   object
 4   FuelType    1336 non-null   object
 5   HP          1436 non-null   object
 6   MetColor    1286 non-null   float64
 7   Automatic   1436 non-null   int64
 8   CC          1436 non-null   int64
 9   Doors       1436 non-null   object
 10  Weight      1436 non-null   int64
dtypes: float64(2), int64(5), object(4)
memory usage: 123.5+ KB

Cleaned Data Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1436 entries, 0 to 1435
Data columns (total 11 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   sno         1436 non-null   int64
 1   Price       1436 non-null   int64
 2   Age         1436 non-null   float64
 3   KM          1421 non-null   float64
 4   FuelType    1436 non-null   object
 5   HP          1430 non-null   float64
 6   MetColor    1436 non-null   float64
 7   Automatic   1436 non-null   int64
 8   CC          1436 non-null   int64
 9   Doors       1436 non-null   object
 10  Weight      1436 non-null   int64
dtypes: float64(4), int64(5), object(2)
memory usage: 123.5+ KB
```

2. Data Transformation - Normalization Normalize the numerical columns such as 'Age', 'KM', 'HP', 'CC', 'Weight'.

```
from sklearn.preprocessing import MinMaxScaler

# Select numeric columns for normalization
numeric_columns = ['Age', 'KM', 'HP', 'CC', 'Weight']

# Apply Min-Max Normalization
scaler = MinMaxScaler()
toyotadf[numeric_columns] =
scaler.fit_transform(toyotadf[numeric_columns])
```

```
# Display normalized data
print("\nNormalized Data:")
print(toyotadf.head())
```

OUTPUT

```
Normalized Data:
   sno  Price       Age        KM FuelType        HP  MetColor  Automatic  \
0    0  13500  0.278481  0.193355   Diesel  0.170732  1.000000          0
1    1  13750  0.278481  0.300149   Diesel  0.170732  1.000000          0
2    2  13950  0.291139  0.171647   Diesel  0.170732  0.674961          0
3    3  14950  0.316456  0.197528   Diesel  0.170732  0.000000          0
4    4  13750  0.367089  0.158433   Diesel  0.170732  0.000000          0

    CC  Doors    Weight
0  1.0  three  0.268293
1  1.0      3  0.268293
2  1.0      3  0.268293
3  1.0      3  0.268293
4  1.0      3  0.276423
```

3. Data Integration Assume we want to integrate additional information, such as a dataset containing car safety ratings.

```
# Example: Assume another dataset with external car features like
safety ratings
additional_data = {
    'sno': [1, 2, 3],    # Replace with actual 'sno' (ID) values
    'Safety_Rating': [5, 4, 3],
    'Maintenance_Cost': [200, 300, 400]
}
additional_df = pd.DataFrame(additional_data)

# Merge with Toyota dataset using the common 'sno' column
toyotadf_integrated = pd.merge(toyotadf, additional_df, on='sno',
how='left')

# Display integrated dataset
print("\nIntegrated Data:")
print(toyotadf_integrated.head())
```

OUTPUT

```
Integrated Data:
   sno  Price       Age        KM FuelType        HP  MetColor  Automatic  \
0    0  13500  0.278481  0.193355   Diesel  0.170732  1.000000          0
1    1  13750  0.278481  0.300149   Diesel  0.170732  1.000000          0
2    2  13950  0.291139  0.171647   Diesel  0.170732  0.674961          0
3    3  14950  0.316456  0.197528   Diesel  0.170732  0.000000          0
4    4  13750  0.367089  0.158433   Diesel  0.170732  0.000000          0

    CC  Doors    Weight  Safety_Rating  Maintenance_Cost
0  1.0  three  0.268293            NaN               NaN
1  1.0      3  0.268293            5.0             200.0
2  1.0      3  0.268293            4.0             300.0
3  1.0      3  0.268293            3.0             400.0
4  1.0      3  0.276423            NaN               NaN
```

Experiment 8: Implementation of Decision Tree algorithm

Aim: Implementation of Decision Tree algorithm

Software Required: Google Co Lab, Jupyter notebook, Kaggle .

Theory:

- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems.

- It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome.**

- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node.** Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.

- **It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.**

- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.

- In order to build a tree, we use the **CART algorithm,** which stands for **Classification and Regression Tree algorithm.**

How does the Decision Tree algorithm Work?

- In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

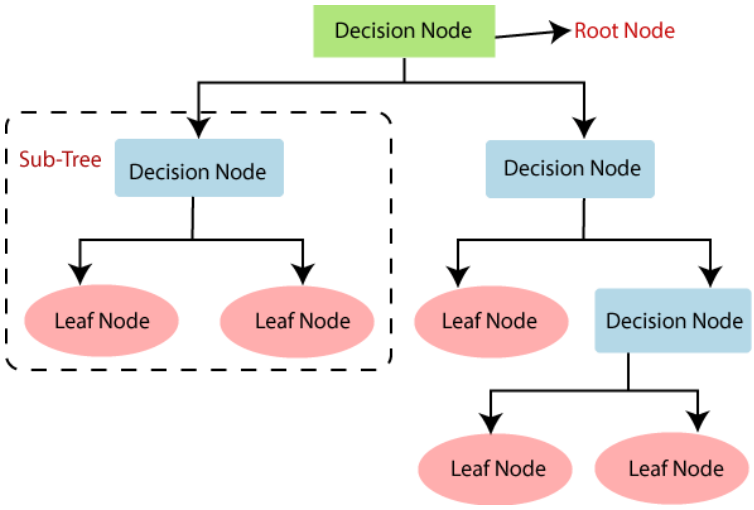
- For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.

- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM).**

- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.

- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

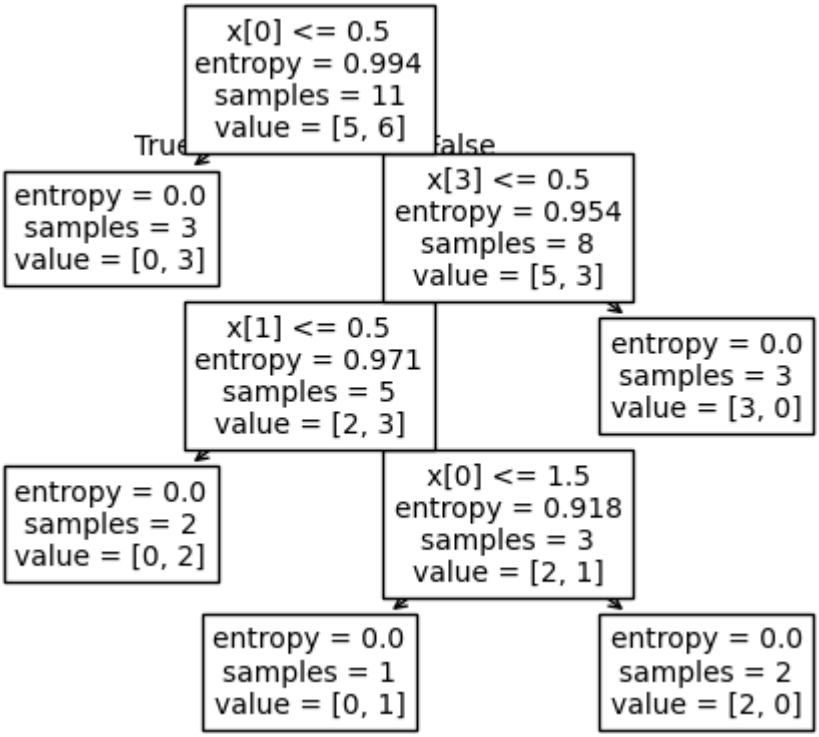


Code:

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
#numpy and pandas initialization
import numpy as np
import pandas as pd
PlayTennis = pd.read_csv("tennis.csv")
#numerical values into numerical values using LabelEncoder
from sklearn.preprocessing import LabelEncoder
Le = LabelEncoder()
PlayTennis['outlook'] = Le.fit_transform(PlayTennis['outlook'])
PlayTennis['temp'] = Le.fit_transform(PlayTennis['temp'])
PlayTennis['humidity'] =
Le.fit_transform(PlayTennis['humidity'])  PlayTennis['windy'] =
Le.fit_transform(PlayTennis['windy']) PlayTennis['play'] =
Le.fit_transform(PlayTennis['play'])
features_cols=['outlook','temp','humidity','windy']
x=PlayTennis[features_cols]
y=PlayTennis.play
from sklearn.model_selection import train_test_split
x_train, x_test, y_train,y_test=train_test_split(x,y,test_size=0.2)
from sklearn.tree import DecisionTreeClassifier
#classifier=DecisionTreeClassifier(criterion='gini')
```

```
classifier=DecisionTreeClassifier(criterion='entropy
')  classifier.fit(x_train,y_train)
DecisionTreeClassifier(criterion='entropy')
classifier.predict(x_test)
classifier.score(x_test,y_test)
from sklearn import tree
clf = tree.DecisionTreeClassifier(criterion = 'entropy')
 clf = clf.fit(x, y)
tree.plot_tree(clf)
```

Output:



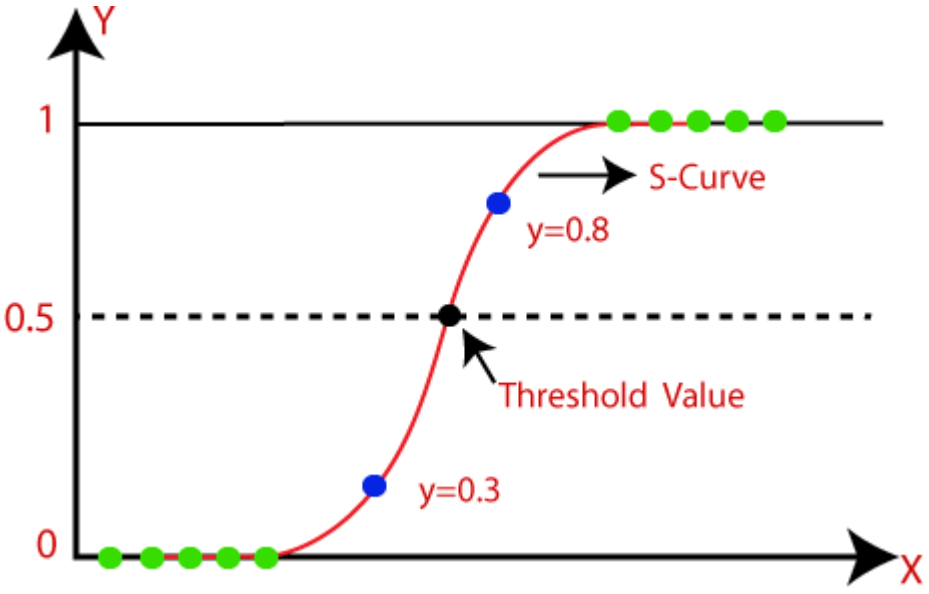
Experiment 9 : Classification of data using logistic regression

Aim: Classification of data using logistic regression

Software Required: Google Co Lab, Jupyter notebook, Kaggle .

Theory:

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1**.
- Logistic Regression is much similar to the Linear Regression except that how they are used. Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems**.
- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc.



Code:

```
#Import required libraries
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O import os
#Read dataset into pandas dataframe.

data=pd.read_csv('Iris.csv')
data.head()
# Display the datatype of each column data.dtypes
# Create a column cat_code which represents the numerical values
for Species column.
data['cat_code']=data.Species.astype('category').cat.codes
data.head()
data.columns
#Select the input features as x
x=data[['SepalLengthCm', 'SepalWidthCm',
'PetalLengthCm','PetalWidthCm']]
#Select the target y=data['cat_code']
data['cat_code']=data.Species.astype('category').cat.codes
#Select the target
y=data['cat_code']

# Splitting train and test datasets
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,sh
uffle=True)
x_train.shape,x_test.shape,y_train.shape,y_test.shape

# Fit the logistic regression model with SKlearn library

from sklearn.linear_model import LogisticRegression
lm=LogisticRegression()
lm.fit(x_train,y_train)


# Accuracy for train dataset
from sklearn.metrics import accuracy_score
# Prediction for test dataset and Accuracy
y_pred = lm.predict(x_test)
```

```
accuracy_score(y_test,y_pred)
```

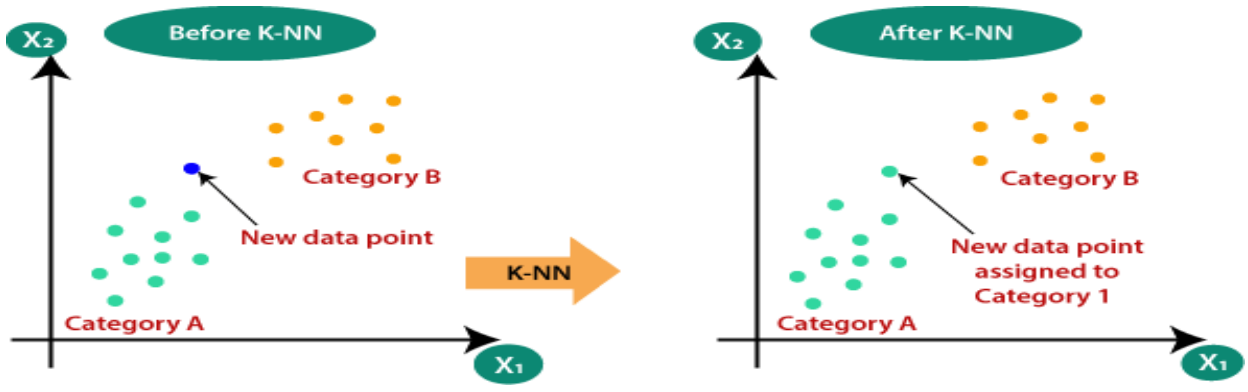
Experiment 10 : Classification of data using K – nearest neighbor approach

Aim: Classification of data using K – nearest neighbor approach

Software Required: Google Co Lab, Jupyter notebook, Kaggle .

Theory:

- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- **K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.**
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.
- **Lazy learning algorithm** − KNN is a lazy learning algorithm because it does not have a specialized training phase and uses all the data for training while classification.
- **Non-parametric learning algorithm** − KNN is also a non-parametric learning algorithm because it doesn't assume anything about the underlying data.



Code:

```
# Import libraries
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g.
pd.read_csv)
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
import seaborn as sns
import matplotlib.pyplot as plt
data=pd.read_csv("Iris.csv")
#data.head()
#data.info()
#data.describe()
#data.head()
#data.tail()
#data.isnull().sum()
data['Species'].value_counts()
data['cat_code']=data.Species.astype('category').cat.codes
#Select the input features as x
x=data[['SepalLengthCm', 'SepalWidthCm',
'PetalLengthCm','PetalWidthCm']]
#Select the target
y=data['cat_code']
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
# Splitting train and test datasets
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,sh
uffle=True)
KNN=KNeighborsClassifier(n_neighbors=3)
KNN.fit(x_train,y_train)
prediction=KNN.predict(x_test)

print('The accuracy of the KNN
is',metrics.accuracy_score(prediction,y_test) *100, 'percent')

#Get accuracy. Note: In case of classification algorithms score
method represents accuracy.
```

```
KNN.score(x_test,y_test)
print("KNN SCORE",KNN.score(x_test,y_test))
#import confusion_matrix
from sklearn.metrics import confusion_matrix
prediction=KNN.predict(x_test)
confusion_matrix(y_test,prediction)
```

Output:

```
The accuracy of the KNN is 93.33333333333333 percent
KNN SCORE 0.9333333333333333
array([[15,  0,  0],
       [ 0,  4,  1],
       [ 0,  1,  9]])
```

Experiment 11 : Implementation of K – means algorithm

Aim: Implementation of K – means algorithm

Software Required: Google Co Lab, Jupyter notebook, Kaggle .

Theory:

K-Means Clustering is an Unsupervised Learning algorithm, which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if K=2, there will be two clusters, and for K=3, there will be three clusters, and so on.

It is an iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.

How does the K-Means Algorithm Work?
The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

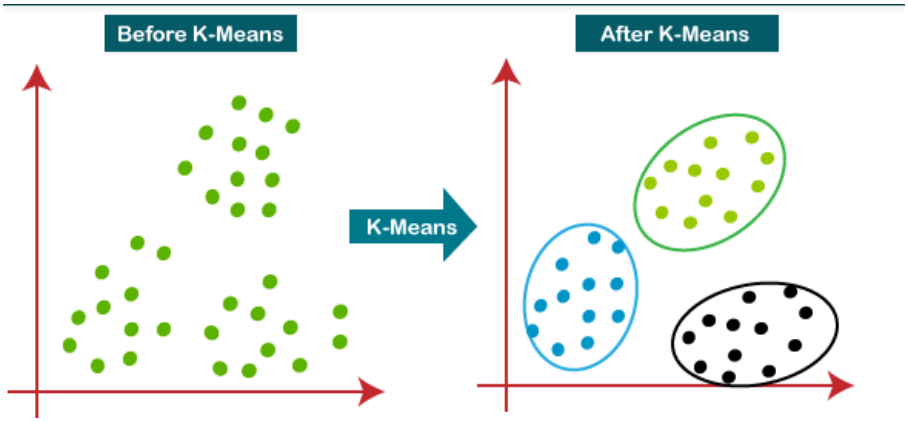
Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.



DATASET: The **make_blobs** function from the sklearn.datasets module is used to generate synthetic data for clustering tasks, especially to test clustering algorithms like K-means.

Key Features of make_blobs:

1. **Creates Clustered Data**: It generates data points grouped into distinct clusters, which makes it useful for testing clustering algorithms.
2. **Customizable**: You can control the number of samples, centers (clusters), dimensionality, standard deviation (spread) of each cluster, and randomness.

Key Parameters:

- **n_samples**: The total number of data points to generate.
- **centers**: Number of clusters (or actual coordinates of the centers) to generate. If an integer is provided, it creates random cluster centers.
- **cluster_std**: Standard deviation (spread) of the clusters. A larger value makes the clusters more dispersed.
- **random_state**: Fixes the random generation of the dataset so you get the same output every time.

Code:

```
#import the KMeans class from the sklearn.cluster module
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Step 1: Generate synthetic data
# X: Contains the coordinates of the generated data points
(features).
# y_: Contains the cluster labels for each data point, indicating
which cluster each point belongs to.
```

```
X, _ = make_blobs(n_samples=300, centers=4,
cluster_std=0.60, random_state=0)

# Step 2: Use the KMeans class from scikit-learn
kmeans = KMeans(n_clusters=4)   # Define the
number of clusters, Creating the KMeans
Object:
kmeans.fit(X)   # Fit the model

# Step 3: Get the
labels and centroids
labels =
kmeans.labels_
centroids = kmeans.cluster_centers_

# Step 4: Visualization of the clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50,
cmap='viridis') plt.scatter(centroids[:, 0],
centroids[:, 1], s=200, c='red', marker='X')
plt.title('K-Means Clustering with scikit-
learn') plt.xlabel('Feature 1')
plt.ylab
el('Feat
ure 2')
plt.show
()
```

Output:

